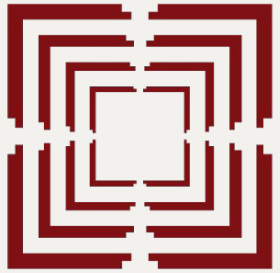


Ahmedabad University

# CSE400 Fundamentals of Probability in Computing

## Milestone-5

## Adaptive Traffic Signal Control using Las Vegas Algorithm

s1_g7_its

# Background and Motivation

- Urban traffic is stochastic and time-varying
- Fixed-time signals assume predictable flow → inefficient

- **Real-World Need:**
  - Congestion, delays, fuel wastage
  - Poor utilization of green signals

- **Course Perspective:**
  - Modeled using probability (Poisson) and queues
  - Requires decision-making under uncertainty

**Motivation**

Need for adaptive + probabilistic control

# Base Paper Overview

- Uses Webster / Modified Webster for cycle time
- Extends to adaptive control using fuzzy logic

- **Key Idea:**
  - Compute cycle time → adjust using real-time data

- **Contribution:**
  - Adaptive signal timing
  - Reduced waiting time and congestion

- **Limitation**
  - Relies on rule-based (fuzzy) adjustment

# Problem Formulation

- **System :**
  - Multi-lane intersection
  - Random vehicle arrivals (Poisson)
  - Each lane = queue
- **Decision Variable:**
  - Green time per lane
- **Objective:**
  - Minimize:
    -Average waiting time
    -Queue length
- **Nature :**
  - Stochastic, time-dependent system
- **Assumptions :**
  - Poisson arrivals
  - Fixed phases, no skipping
  - Constant saturation flow

# Mathematical Modeling

| Random Variable | Meaning | Distribution | Units |
|---|---|---|---|
| $A_i$ | Vehicle arrivals | Poisson($\lambda$) | vehicles/time-step |
| $\delta_i$ | Green time adjustment | Uniform (biased) | multiplier |
| $Q_i$ | Queue length | Stochastic (derived) | vehicles |
| $W_i$ | Waiting time | Stochastic (derived) | time-step |

# Mathematical Modeling

- **Vehicle arrivals :**
  - Vehicle arrivals are very random in nature
    $A_i(t) \sim \text{Poisson}(\lambda_i)$
    where,
    $\lambda_i$ : average arrival rate per lane

- **Queue length :**
  - Each lane behaves like an individual queue.
    $Q_i(t+1) = Q_i(t) + A_i(t) - D_i(t)$
    where,
    $Q_i(t)$ : number of vehicles in lane $i$ at time $t$
    $A_i(t)$ : arrivals
    $D_i(t)$ : departures

# Mathematical Modeling

- **Waiting time approximation :**

$$W_i = \frac{Q_i}{\lambda_i}$$

- **Green time adjustment :**

$$g_i = \alpha_i \cdot C_e + g_{\min}$$

- **Objective function :**

$$\min \frac{1}{N} \sum W_i$$

# Deterministic Approach

- A **deterministic approach** is a modeling or computational method where the same input always produces the exact same output, with no randomness or uncertainty.

- **Baseline Strategy :**
  - Use fixed mathematical formulas (Webster / Modified Webster)
  - Inputs are treated as exact values (no randomness in decision logic)
  - Predefined cycle length and green splits computed analytically
  - Same decision rule applied on every simulation run

- **Cycle time formula :**

$$C_o = \frac{1.5L + 5}{1 - \sum y_i}$$

where,

$$y_i = \frac{\text{flow}}{\text{saturation}}$$

# Randomized Approach (Las Vegas)

- A **Las Vegas algorithm** will always produce the correct answer.
- **Core Ideation :**
  - Introduce controlled randomness over deterministic solution.
  - Enable exploration of better signal allocations.
- **Methodology :**
  - Initialize with Webster-based green times.
  - Generate multiple candidate solutions.
  - Apply biased perturbation to each lane.

$$g_i' = g_i \cdot \delta_i$$

$$\delta_i \sim \text{Uniform (biased)}$$

- **Selection strategy :**
  - Evaluate all candidate configurations.
  - Choose the one with minimum average waiting time.

# Coding and Simulation

- **End-to-End System Pipeline :**
  - SUMO (Simulation of Urban Mobility) as a simulator of the traffic intersection.
  - Python is used for the logic implementation and backend control.
  - TraCI (Traffic Control Interface) is used as a connection between Python and SUMO.

# Coding and Simulation

- **SUMO :**
  - "Simulation of Urban MObility" (SUMO) is an open source, highly portable, microscopic and continuous traffic simulation package designed to handle large networks.
  - It allows for intermodal simulation including pedestrians and comes with a large set of tools for scenario creation.

- **TraCI :**
  - TraCI is the short term for "Traffic Control Interface". Giving access to a running road traffic simulation.
  - It allows to retrieve values of simulated objects and to manipulate their behavior "on-line".

# Coding and Simulation

```python
def deterministic_step(Q):

    arrivals_list = []
    yi_list = []
    cycle_time = 60

    for i in range(LANES):
        arrivals = sample_arrivals(lambda_i[i])
        arrivals_list.append(arrivals)

        flow = arrivals * 3600.0 / max(cycle_time, 1e-6)
        yi = flow / 1800.0
        yi_list.append(max(yi, 1e-6))

    y_sum = sum(yi_list)
    L = LANES * 3.0

    Co = (1.5 * L + 5.0) / max((1 - y_sum), 1e-6)
    Co = max(40.0, min(Co, 120.0))

    greens = []
    Ce = max(Co - L - LANES * 10.0, 0.0)

    for yi in yi_list:
        alpha = yi / y_sum if y_sum > 1e-9 else 1.0 / LANES
        gi = alpha * Ce + 10.0
        greens.append(gi)

    new_Q = []
    total_wait = 0

    for i in range(LANES):

        arrivals = arrivals_list[i]

        service = mu_i[i] * (greens[i] / Co)
        departures = min(Q[i], int(service * Co))

        Qi = max(Q[i] + arrivals - departures, 0)
        Wi = Qi / max(lambda_i[i], 1e-6)

        total_wait += Wi
        new_Q.append(Qi)

    return new_Q, total_wait / LANES
```

**Implementation of deterministic traffic signal control using Webster's cycle time formulation**

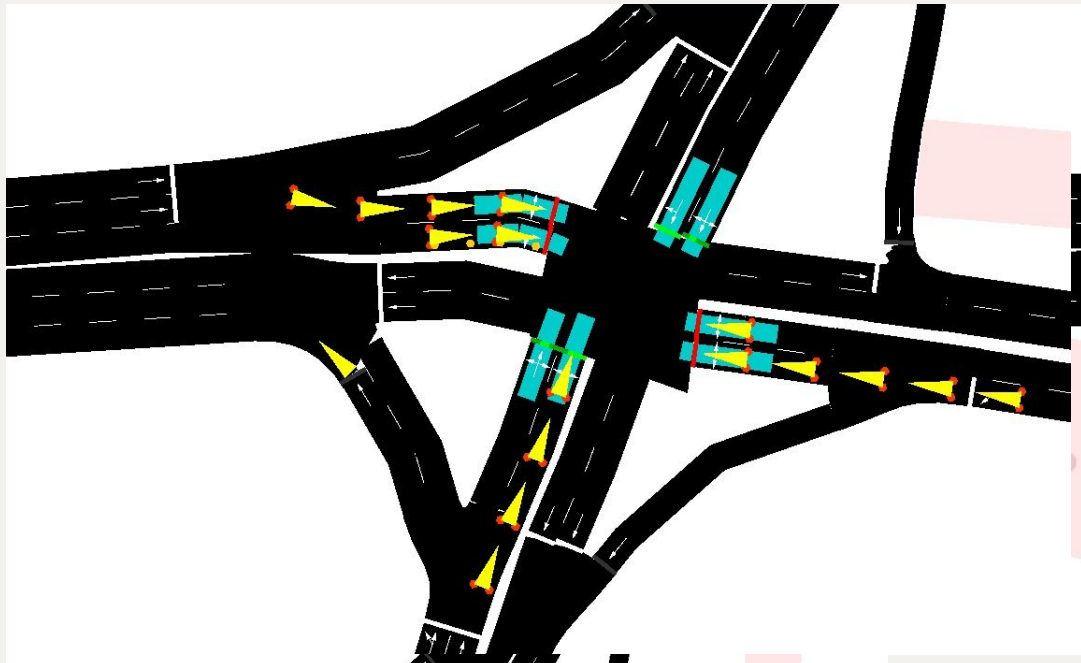
# Coding and Simulation

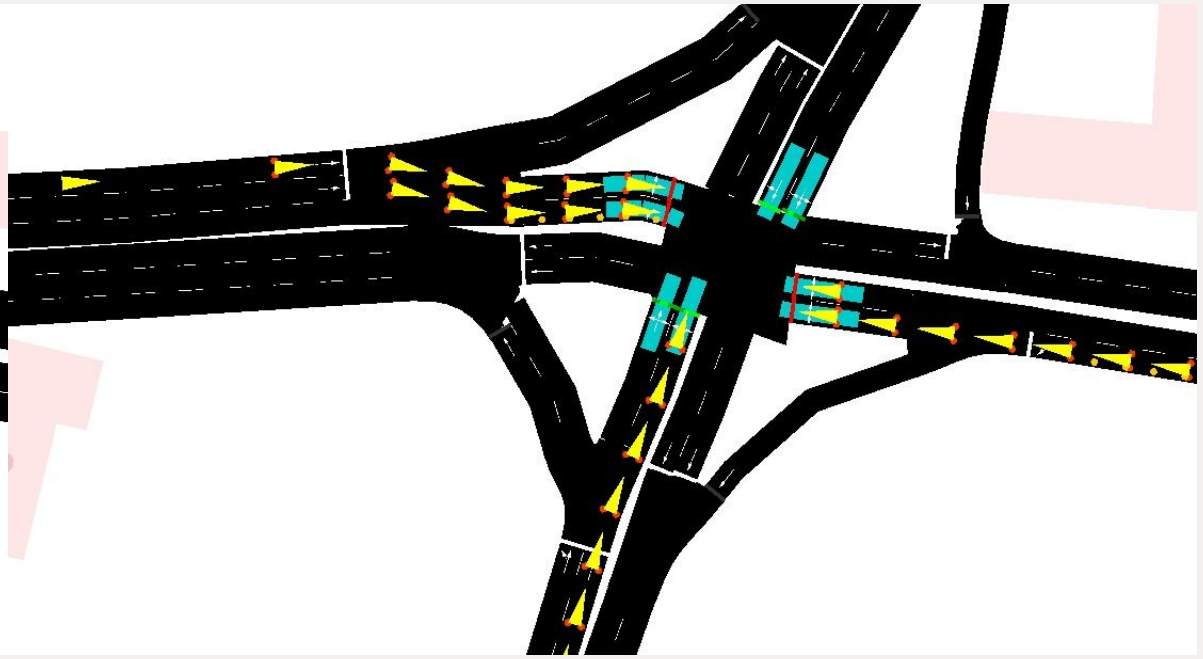
```python
def las_vegas_step(Q, trials=50):

    # Baseline
    det_Q, det_wait = deterministic_step(Q)

    arrivals_list = []
    yi_list = []
    cycle_time = 60

    for i in range(LANES):
        arrivals = sample_arrivals(lambda_i[i])
        arrivals_list.append(arrivals)

        flow = arrivals * 3600.0 / max(cycle_time, 1e-6)
        yi = flow / 1800.0
        yi_list.append(max(yi, 1e-6))

    y_sum = sum(yi_list)
    L = LANES * 3.0

    Co = (1.5 * L + 5.0) / max((1 - y_sum), 1e-6)
    Co = max(40.0, min(Co, 120.0))

    base_greens = []
    Ce = max(Co - L - LANES * 10.0, 0.0)

    for yi in yi_list:
        alpha = yi / y_sum if y_sum > 1e-9 else 1.0 / LANES
        gi = alpha * Ce + 10.0
        base_greens.append(gi)

    Q_array = np.array(Q)
    weights = Q_array / max(np.sum(Q_array), 1e-6)

    best_Q = det_Q
    best_wait = det_wait
    best_score = det_wait
    best_deltas = [1]*LANES
```

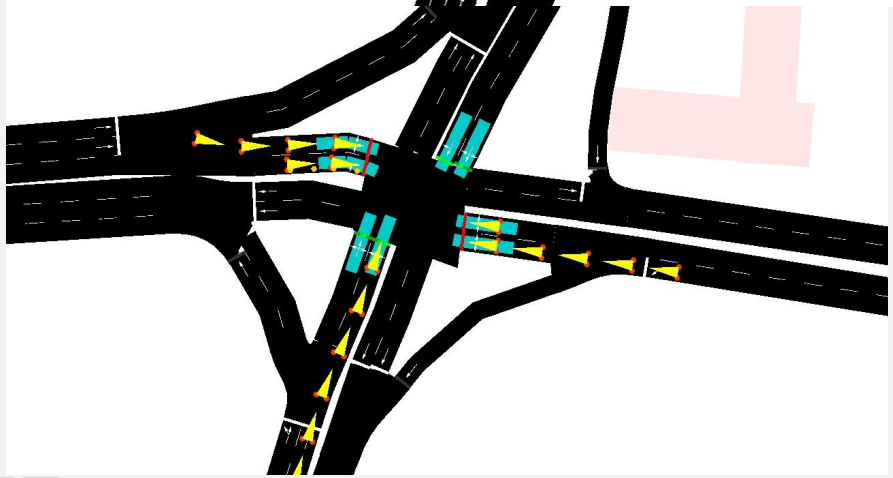
```python
for _ in range(trials):

    trial_Q = []
    total_wait = 0
    deltas = []

    for i in range(LANES):

        bias = 1 + 0.8 * weights[i]
        noise = random.uniform(0.8, 1.2)

        delta = max(0.7, min(bias * noise, 1.8))
        deltas.append(delta)

        gi = base_greens[i] * delta

        service = mu_i[i] * (gi / Co)
        departures = min(Q[i], int(service * Co))

        arrivals = arrivals_list[i]

        Qi = max(Q[i] + arrivals - departures, 0)
        Wi = Qi / max(lambda_i[i], 1e-6)

        total_wait += Wi
        trial_Q.append(Qi)

    avg_wait = total_wait / LANES
    avg_queue = sum(trial_Q) / LANES

    score = 0.7 * avg_wait + 0.3 * avg_queue

    if (score < best_score) and (avg_wait <= det_wait):
        best_score = score
        best_wait = avg_wait
        best_Q = trial_Q
        best_deltas = deltas

if best_wait > det_wait:
    return det_Q, det_wait, [1]*LANES

return best_Q, best_wait, best_deltas
```

**Randomized optimization with Las Vegas Algorithm ensuring improvement over deterministic baseline**

# Coding and Simulation

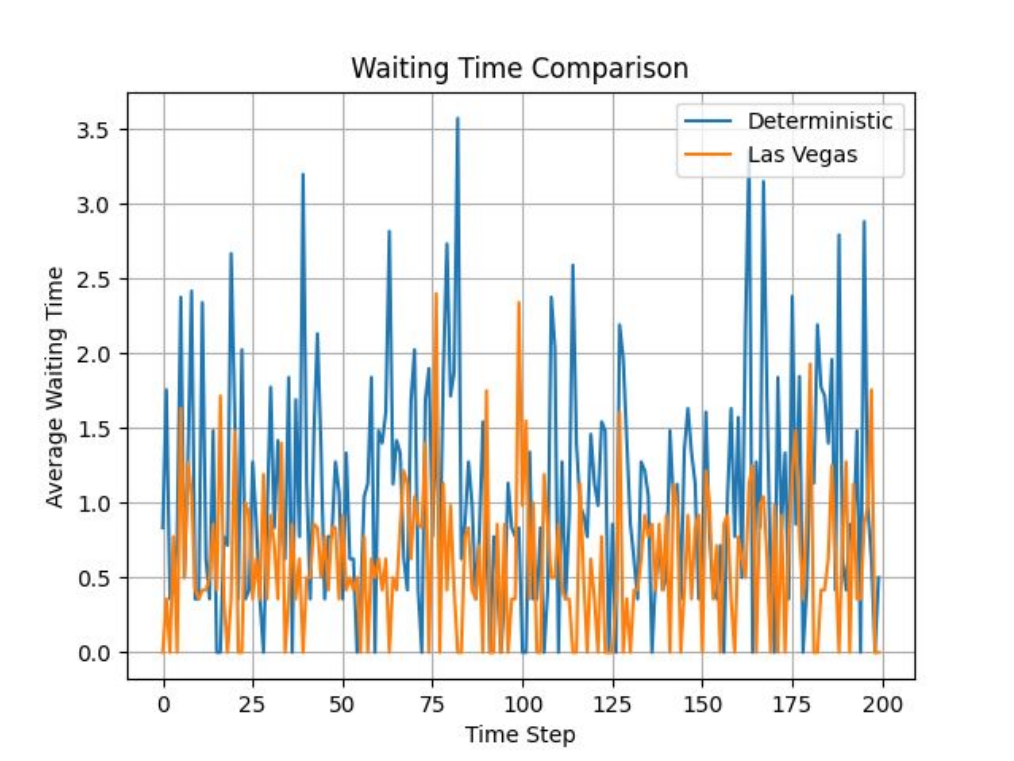


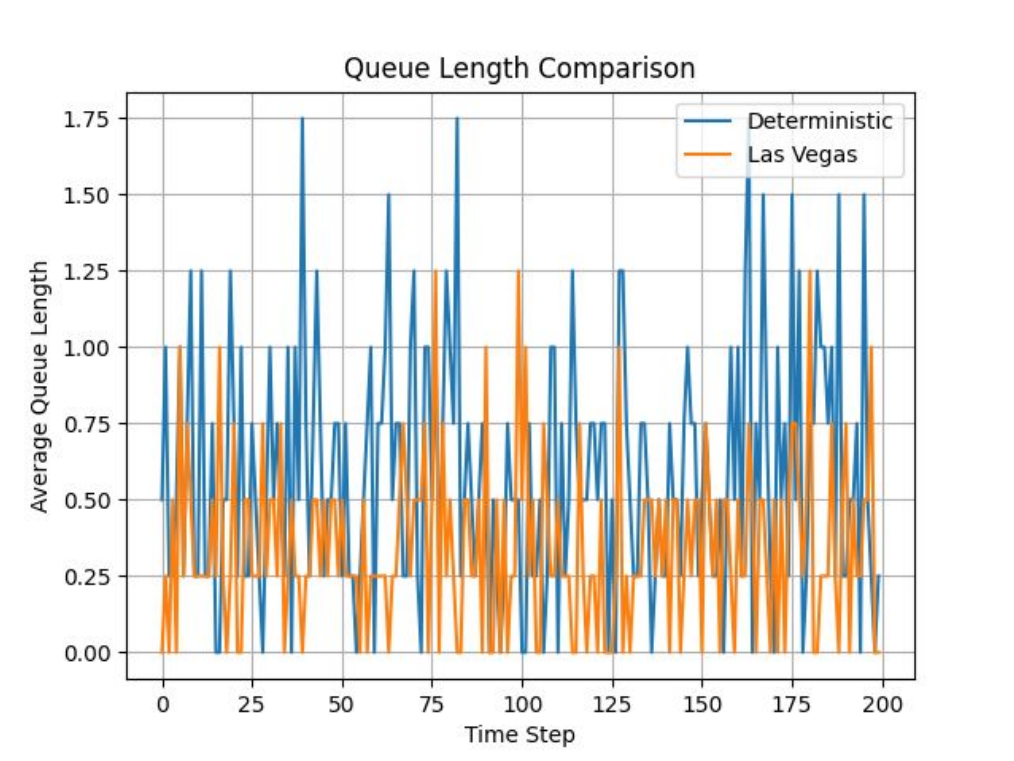




**Visualization of real-time vehicle movement and traffic flow at an intersection in SUMO**

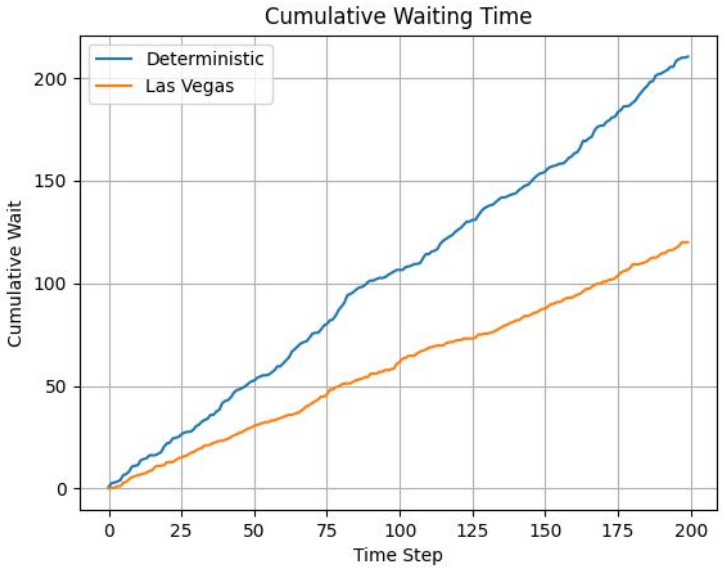
# Results

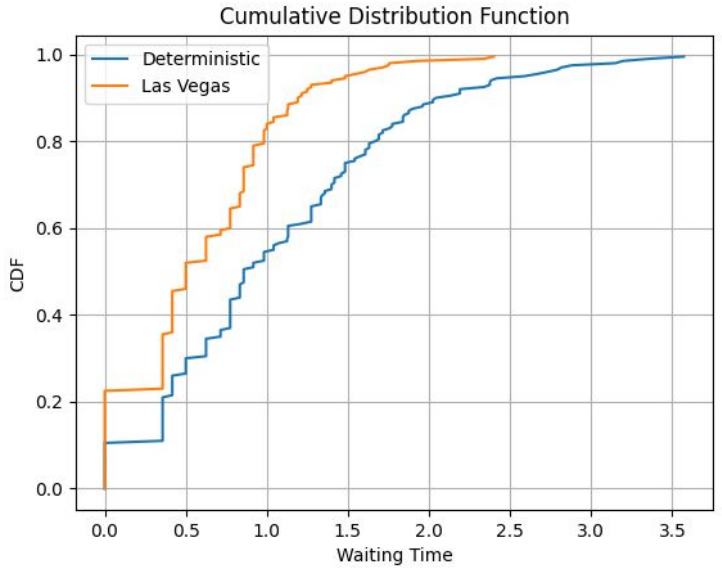


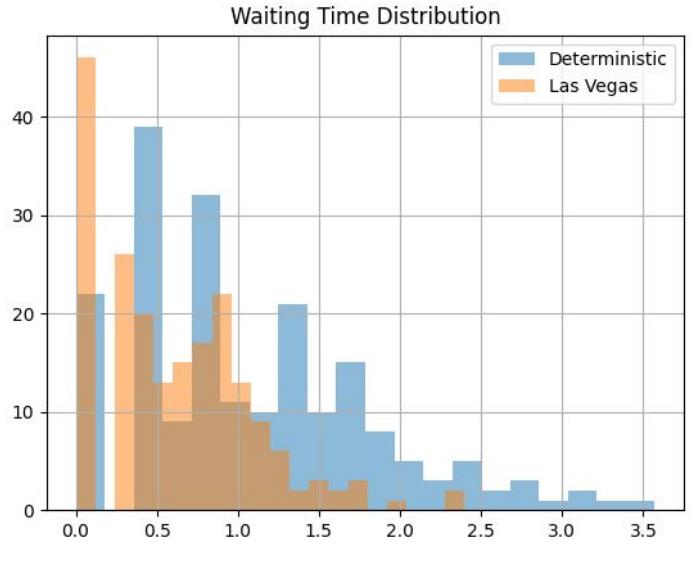




**Comparative analysis shows reduced waiting time and queue length using randomized optimization**

# Results




**Improved CDF and distribution indicate faster service and reduced congestion**

# Inference – Domain Perspective

- **Traffic as a real world system :**
  - Intersections behave like stochastic queuing systems.
  - Vehicle arrivals are completely random and time-varying.
  - Peak hour congestions result into non-linear queue buildup.

- **Limitations of deterministic control :**
  - It assumes fixed time arrival inputs.
  - It can't adopt to sudden traffic bursts.
  - As a result, there are
    - Idle green signals
    - Increased waiting time

- **Advantages of Las Vegas approach :**
  - Use of controlled randomness
  - Effective handling of real-time fluctuations.
  - Dynamically adjustments of signal timing.

- **System level Improvements :**
  - Waiting time ↓
  - Queue length ↓
  - Throughput ↑
  - Signal efficiency ↑

**Traffic is stochastic → Control must be adaptive, not fixed.**

# Inference – Computer Science Perspective

- **System Characteristics :**
  - Stochastic optimized problem.
  - Real-time decision making system.

- **Algorithmic Insight :**
  - Deterministic approach → ensures baseline performance.
  - Las Vegas approach → enables exploration and improvement.

- **Key Properties :**
  - Online algorithm (No prior training required).
  - State-dependent decision logic.
  - Hybrid deterministic-randomized framework.

# References

Harchol-Balter, M. (2024). Introduction to probability for computing. Cambridge University Press. March 25, 2026, https://www.cs.cmu.edu/~harchol/Probability/chapters/chpt21.pdf

M. E. M. Ali, A. Durdu, S. A. Celtek and A. Yilmaz, "An Adaptive Method for Traffic Signal Control Based on Fuzzy Logic With Webster and Modified Webster Formula Using SUMO Traffic Simulator," in IEEE Access, vol. 9, pp. 102985-102997, 2021, doi: 10.1109/ACCESS.2021.3094270.

https://sumo.dlr.de/docs/index.html

https://sumo.dlr.de/docs/TraCI/index.html

Rushin Modi - AU2440028
Philip Ryan Rocha - AU2320182
Naisha Dave - AU2440181
Aditi Kapadiya - AU2440139
Viranshul Panchal - AU2440209